Interpretable Non-negative Matrix Factorization for Nutrition-Aware Recipe Recommendation

Patrick Gomes de Oliveira

October 15, 2026

Abstract

We evaluate a from-scratch, multiplicative-update Non-negative Matrix Factorization (NMF) for top-N Food.com recipe recommendation under a strict global temporal split, combined with a post-hoc nutrition-aware filter over calories, sodium, and allergens. Unlike signed factorizations or pairwise-ranking embeddings, NMF constrains both factor matrices to be non-negative, yielding additive, interpretable latent components. The system includes point-in-time features, strong non-neural baselines, reproducible Floci/AWS infrastructure, and a measurement of how much the nutrition filter changes the ranking.

1 Introduction

Recipe recommendation differs from movie or anime recommendation in one structural way: every candidate carries hard, user-specific domain constraints – calories, sodium, allergens – that a purely collaborative ranking has no mechanism to respect. A model that ranks a peanut-laden dessert first for a user with a peanut allergy is not merely suboptimal, it is unsafe. This motivates pairing a classical collaborative-filtering model with an explicit, independently auditable nutrition-aware post-filter, rather than hoping the ranker implicitly learns dietary restrictions from sparse review data.

We choose Non-negative Matrix Factorization (NMF) as the collaborative component, in contrast to the signed truncated SVD used for temporal movie ranking and the pairwise BPR embeddings used for anime ranking in companion projects of this portfolio. NMF constrains both factor matrices to be element-wise non-negative, which forces every latent component to be an additive, non-negative combination of recipes rather than a direction that can cancel against other directions. The practical consequence is interpretability: a column of the item-factor matrix reads as a non-negative "flavor/ingredient profile" – a recipe either participates in a profile or it does not, and a user's affinity to it is never used to cancel out affinity to another profile. Signed factorizations cannot offer this reading, since a large positive loading on one factor can be entirely compensated by a large negative loading elsewhere.

Food.com reviews, like MovieLens ratings, carry real event timestamps. We therefore follow the temporal-split convention established for time-aware recommendation in this portfolio: the primary evaluation protocol is a global wall-clock cut-off, with leave-last-one-out reported as a secondary protocol for comparability with the per-user evaluation most published notebooks use. All feature engineering is point-in-time correct under the primary protocol, and the gap between the two protocols is reported rather than hidden.

2 Dataset and Split

Food.com Recipes and Reviews pairs a recipe catalogue (name, category, ingredients, and nutrition facts: calories, fat, sodium, protein, carbohydrate, sugar) with roughly 1.4M user reviews carrying a real Unix timestamp. Reviews of at least four (on the platform's 0–5 scale) define positive implicit interactions; lower ratings are dropped rather than treated as negatives, since they carry strictly more signal than an unobserved interaction. An iterative five-core filter removes users and recipes without enough evidence on either side, repeated until the filtered set is stable.

If τ_1 and τ_2 are the 80th and 90th percentiles of the review timestamp distribution, training uses $t < \tau_1$, validation uses $\tau_1 \leq t < \tau_2$, and test uses $t \geq \tau_2$. Evaluation rows for users absent from training are dropped and the resulting cold-user rate is reported rather than silently folding those users into the popularity prior. All feature transformations are fitted on the training partition only. Leave-last-one-out is also implemented and reported as a secondary protocol: it holds out each user's final two chronological interactions regardless of the global time axis, and is the convention most published recipe-recommendation notebooks use.

3 Feature Engineering

Features are declared in `conf/features.yaml` and implemented in `src/recommender/features/`, following the same declarative registry pattern used throughout this portfolio: a feature cannot be computed without being declared, and a declaration cannot go unimplemented.

Table 1: Engineered features. Every aggregate is computed point-in-time: only interactions strictly before the prediction cut-off contribute to its value.

Feature	Family	Transform	Point-in-time
`user_activity_count`	user	`user_activity_count`	–
`user_mean_rating`	user	`user_mean_rating`	–
`user_tenure_days`	user	`user_tenure_days`	–
`user_recency_days`	context	`user_recency_days`	✓
`item_popularity_rank`	item	`item_popularity_rank`	–
`item_trailing_popularity`	statistical	`item_trailing_popularity`	✓
`item_category_tokens`	text	`item_category_tokens`	–
`item_calories`	item	`item_calories`	–
`item_sodium_mg`	item	`item_sodium_mg`	–

Every point-in-time feature receives an explicit cut-off timestamp and is covered by adversarial leakage tests that forge future interactions, compare cut-offs, and scramble the training association to confirm the pipeline cannot read what it should not see.

User features capture activity volume, average rating generosity, taste tenure, and recency relative to the cut-off. Recipe features capture catalogue-wide and trailing popularity, category, and – distinct from the companion MovieLens and anime projects – a direct passthrough of nutrition facts (calories, sodium). The nutrition fields are deliberately excluded from the collaborative model's training matrix: they are consumed only by the post-hoc filter described in Section 4, so the dietary constraint remains independent of whatever the factorization has learned from review co-occurrence.

`user_activity_count` Review volume separates a user with enough history for collaborative signal from one riding on the popularity prior. The distribution is heavy-tailed, so it is log-compressed before NMF's input matrix or any downstream linear model consumes it.

`user_mean_rating` Implicit filtering keeps only positives, so this measures how generously a user rates recipes they already liked. It calibrates the confidence attached to a positive rather than the positive itself.

`user_tenure_days` Calendar span between a user's first and last training review. Distinguishes fifty reviews spread over three years from fifty reviews in one weekend; the two imply very different taste stability.

`user_recency_days` Days from a user's last training review to the cut-off. A dormant user's stored preferences are staler than an active user's, and the serving path should weigh them accordingly. Depends on the cut-off, so it is recomputed per partition and never cached across splits.

`item_popularity_rank` Normalized dense popularity rank over the full recipe catalogue. Backs the popularity baseline and the cold-start fallback. Total over the catalogue by construction, so serving never hits a missing key.

`item_trailing_popularity` Review count in the 90 days before the cut-off. Lifetime popularity is dominated by whatever recipe went viral years ago; a trailing window tracks what is popular now, which is what a system serving traffic at the cut-off actually needs. This is the feature that justifies evaluating on a temporal split.

`item_category_tokens` Recipe category label (Dessert, Main Dish, Salad, ...). Content signal available for recipes with no interactions at all, which is the only thing a purely collaborative model cannot supply for a cold recipe.

`item_calories` Calories per serving, passed through from the recipe catalogue. Feeds the nutrition-aware post-filter directly rather than the NMF ranker, so the domain constraint stays independent of whatever the collaborative model has learned.

`item_sodium_mg` Sodium content in milligrams, passed through from the catalogue. Together with calories it is the minimal nutrition signal the post-filter needs to enforce hard dietary constraints on the ranked candidate list.

4 Methodology

Let $X \in \mathbb{R}_{\geq 0}^{|\mathcal{U}| \times |\mathcal{I}|}$ be the non-negative training interaction matrix (binary in this work: $X_{ui} = 1$ if user u positively reviewed recipe i before the cut-off, else 0). Non-negative Matrix Factorization seeks two non-negative factor matrices $W \in \mathbb{R}_{\geq 0}^{|\mathcal{U}| \times k}$ and $H \in \mathbb{R}_{\geq 0}^{k \times |\mathcal{I}|}$ such that

$$X \approx WH, \qquad W \geq 0,\ H \geq 0,$$

by minimizing the squared Frobenius reconstruction error

$$\min_{W,H \geq 0} \|X - WH\|_F^2.$$

We implement Lee and Seung's multiplicative update rules [1] directly in NumPy rather than via automatic differentiation or a gradient-descent optimizer. Each sweep applies, in order,

$$H \leftarrow H \odot \frac{W^\top X}{W^\top W H}, \qquad W \leftarrow W \odot \frac{X H^\top}{W H H^\top},$$

where $\odot$ and the division are element-wise and a small additive constant stabilizes the denominator against exact zeros. Both updates are non-negative, element-

wise ratios of non-negative quantities applied to the already non-negative current factors, so W and H remain non-negative at every sweep without an explicit projection step. Lee and Seung show that the Frobenius objective is non-increasing under these updates [2]; we verify this empirically for our implementation in `tests/test_models.py` by tracking the reconstruction error across iterations and asserting it never increases.

The recommendation score for user u and recipe i is $\hat{y}_{ui} = W_{u:}H_{:i}$. Recipes already reviewed by u in training are removed before top-N selection; a user absent from training receives the catalogue popularity ordering as a cold-start fallback.

4.1 Interpretability

Because every entry of W and H is non-negative, a column of H can only add to, never subtract from, a recipe's reconstructed score. Each latent component is therefore a non-negative mixture of recipes – legible as a "flavor/ingredient profile" – and each user is a non-negative mixture of such components. A signed factorization (PureSVD) admits no such reading: a large positive loading on one latent direction may be entirely cancelled by a large negative loading on another, so individual factors do not correspond to anything a person would recognize as a coherent taste profile.

4.2 Nutrition-aware post-filter

Let $R_u = (i_1, \ldots, i_m)$ be the ranked candidate list the factorization produces for user u, ordered by $\hat{y}_{ui_1} \geq \cdots \geq \hat{y}_{ui_m}$. The nutrition filter is a hard, order-preserving post-filter: given per-recipe calorie, sodium, and allergen-ingredient attributes a_i and a constraint set $\mathcal{C}$ (e.g. maximum calories, maximum sodium, excluded allergen tokens), it produces the surviving sublist

$$R'_u = \big(i_j \in R_u : i_j \text{ does not violate } \mathcal{C}\big),$$

preserving the relative order of survivors. R'_u is truncated to the final list length k after filtering, so a sufficiently large candidate pool must be requested from the factorization to leave k survivors after filtering. Because the filter reads a_i directly from the recipe catalogue rather than from anything the factorization produced, a constraint holds regardless of what the collaborative signal has learned, and the filter's effect on ranking quality can be measured in isolation (Section 6).

5 Experiments

We benchmark three models against the primary temporal split: the hand-written multiplicative-update NMF ($k = 16$ latent factors), a shrunk cosine item-kNN baseline, and a non-personalised popularity baseline. Ranking quality is measured at $K = 10$ with Recall@K, NDCG@K, Mean Average Precision at K (MAP@K), and catalogue coverage: the fraction of recipes that appear in at least one user's top-K list across the evaluated population. Candidates for every model are the full recipe catalogue minus the user's training history.

To measure the nutrition filter's effect in isolation from any ranking quality difference between models, we fix the model to NMF, request a candidate pool of 50 recipes per user, apply the filter (`max_calories=450`, `max_sodium_mg=600`, reflecting a "light meal, moderate sodium" profile), and compare NDCG@10 and Recall@10 on the first 10 survivors against the unfiltered first 10 candidates from the same pool. We additionally report the mean fraction of the 50-recipe candidate pool that the filter removes, which is the direct cost of enforcing the constraint.

All numbers in Section 6 are produced by `scripts/export_results_table.py` against the synthetic fixture described in `scripts/download_data.py`, run with no network access or Kaggle credentials, so the pipeline, this paper, and the CI workflow all execute identically.

6 Results

Table 2: Global temporal test results on the reproducible fixture.

Model	NDCG@10	Recall@10	MAP@10	Coverage
NMF	0.1270	0.1497	0.0719	0.3507
ItemKNN	0.1378	0.1969	0.0799	0.4970
Popularity	0.1040	0.1637	0.0549	0.0301

Table 3: Nutrition filter impact on NMF (k=10, 50-recipe candidate pool).

Quantity	Value
Mean fraction of candidates removed	0.7066
NDCG@10 unfiltered	0.1270
NDCG@10 filtered	0.0785
NDCG@10 delta	-0.0485

ItemKNN is strongest on the fixture; NMF substantially improves over the popularity floor but does not beat the tuned neighbourhood method on raw ranking quality. This negative result is retained deliberately: the point of this portfolio entry is interpretability and a principled nutrition constraint, not a claim that NMF dominates every classical baseline.

The nutrition filter removes the large majority of the 50-recipe candidate pool under the (realistic, moderately strict) 450-calorie/600-mg-sodium constraint used here, and the resulting NDCG@10 drop quantifies the cost of enforcing that constraint rather than leaving it hidden inside an opaque re-ranking step. A looser constraint removes fewer candidates and costs less ranking quality; both numbers are reported by

`scripts/export_results_table.py` for any constraint setting, so the trade-off is inspectable rather than asserted.

7 Deployment

Training writes factors and feature tables to S3 and publishes precomputed, already nutrition-filtered top-N lists to DynamoDB. Lambda performs a bounded key lookup. EventBridge schedules nightly retraining and CloudWatch receives logs. The default Terraform profile targets the local Floci AWS emulator and therefore requires no paid AWS account.

8 Limitations and Conclusion

NMF is additive and inexpensive, and we have shown its latent components admit a non-negative "flavor profile" reading that signed factorizations cannot offer, but it does not model temporal drift inside the factorization and offers only a popularity fallback for cold users, the same limitation PureSVD carries in the companion MovieLens project. The nutrition filter is a hard post-filter rather than a learned constraint, which is a deliberate safety choice but means its interaction with ranking quality must be measured explicitly, as we do in Section 6, rather than assumed away. Results on the synthetic fixture are not claims about the full 1.4M-review Food.com dataset. The project demonstrates a leakage-resistant, tested classical recommendation pipeline that combines an interpretable collaborative model with an auditable domain constraint, rather than a claim that NMF or any single nutrition-filter setting is optimal.

References

[1] Daniel D. Lee and H. Sebastian Seung. Learning the parts of objects by non-negative matrix factorization. *Nature*, 401:788–791, 1999.

[2] Daniel D. Lee and H. Sebastian Seung. Algorithms for non-negative matrix factorization. In *Advances in Neural Information Processing Systems*, volume 13, 2001.